

CS 406 – Web Programming

Hassan Sartaj



National University
Of Computer and Emerging Sciences



MEAN STACK



Mongo DB
(database system)



Express
(back-end web
framework)



Angular.js
(front-end
framework)

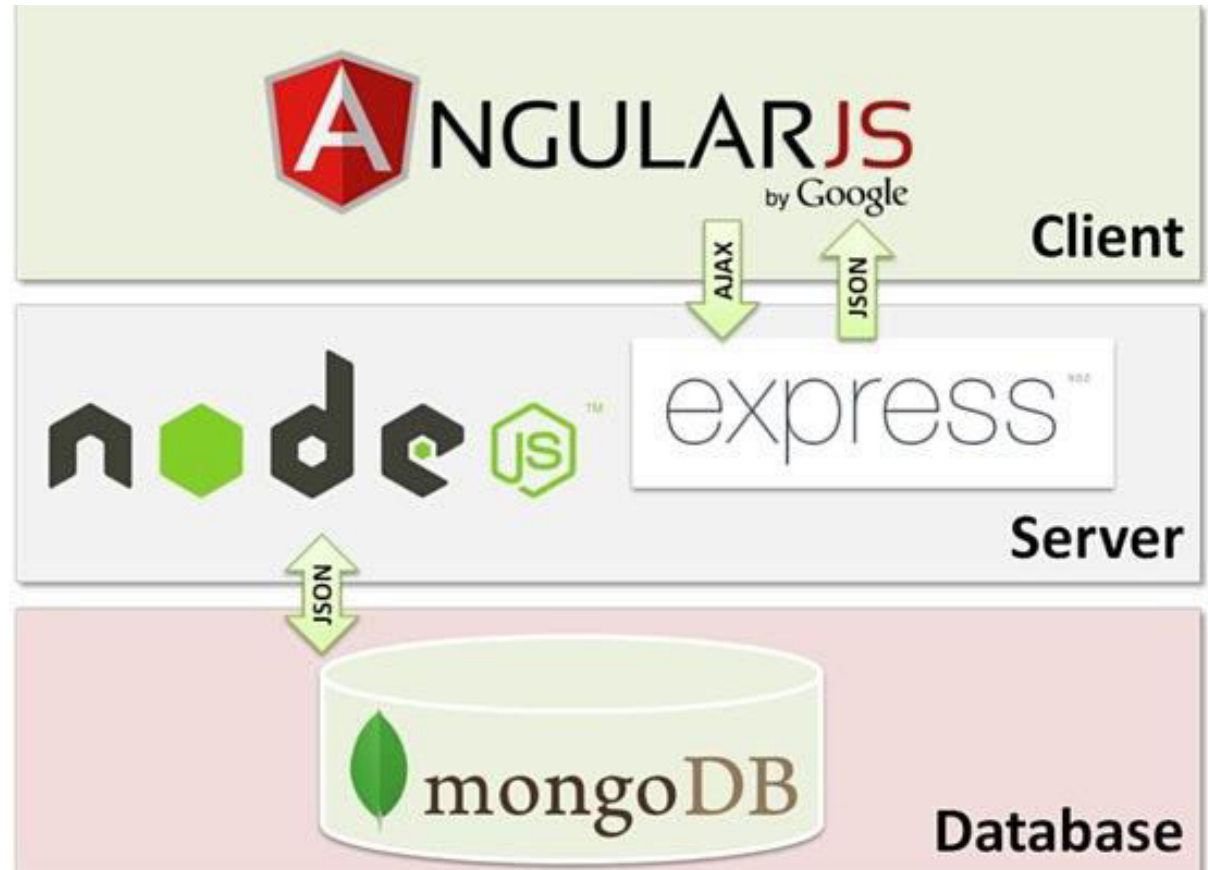


Node.js
(back-end runtime
environment)

Collection of JavaScript based technologies used to develop web applications.

MEAN

- M = MongoDB
- E = Express
- A = Angular.js
- N = Node.js



Setting up MEAN Development Environment

- **MongoDB**

- Download the latest *MongoDB Community Server* from [MongoDB](#), and follow the [installation guide](#) for your platform (e.g. [Windows](#), [MacOS](#), [Ubuntu](#)).

- **Node.js and NPM Packages**

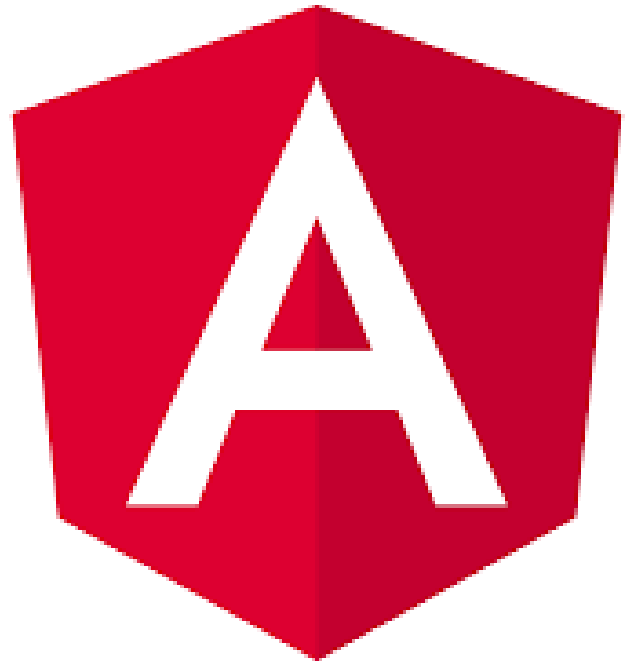
- Download and install *Node.js 14.x* from [nodejs.org](#). We will also need the following global packages:
 - `express-generator` for creating web applications using the Express framework.
 - `@angular/cli` for creating and building Angular projects.
 - `nodemon` for running and monitoring Node.js applications.
- To install a package globally (meaning the package will be available to all projects instead of being used by only one project), open a command prompt and run the following command:
 - `npm install -g <package_name>` : For example, to install nodemon, simply run "`npm install -g nodemon`".

- **Text Editors for Developers**

- [Sublime Text](#), [Visual Studio Code](#), and [Atom](#) are all great code editors and popular among MEAN stack developers. Just pick one that you like the most

AngularJS

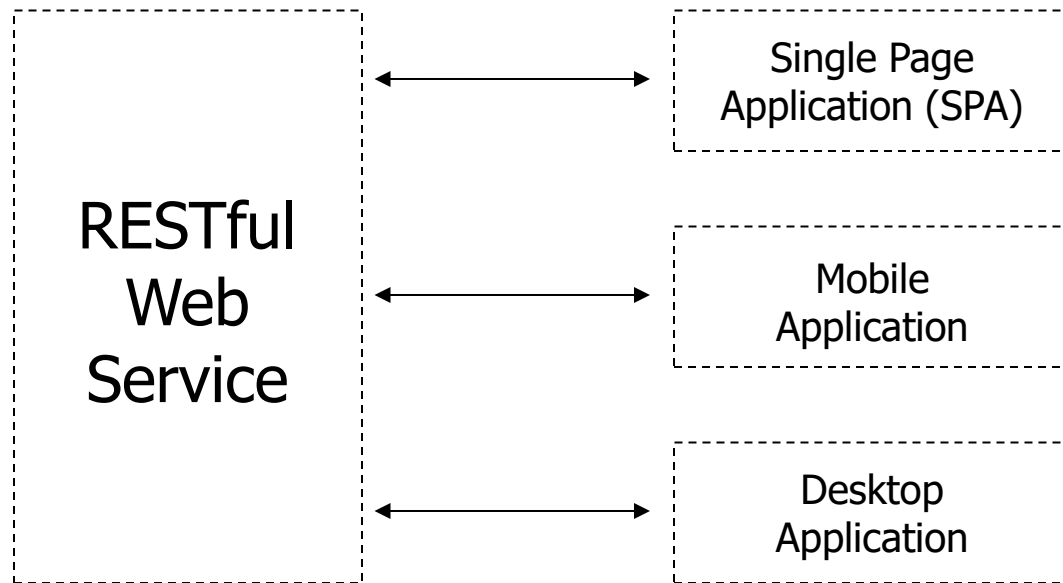
Client Side Application
Framework



Introduction

- **Angular** JS (commonly referred to as "Angular" is an open-source web application framework
- Mainly maintained by
 - Google
 - Community of individuals and corporations
- Simplified front-end Development
- One platform for Desktop & Mobiles
- Address many of the challenges encountered in developing single-page applications
- Useful as structural framework for dynamic web apps.

Angular: An Application Framework for the Client-Side

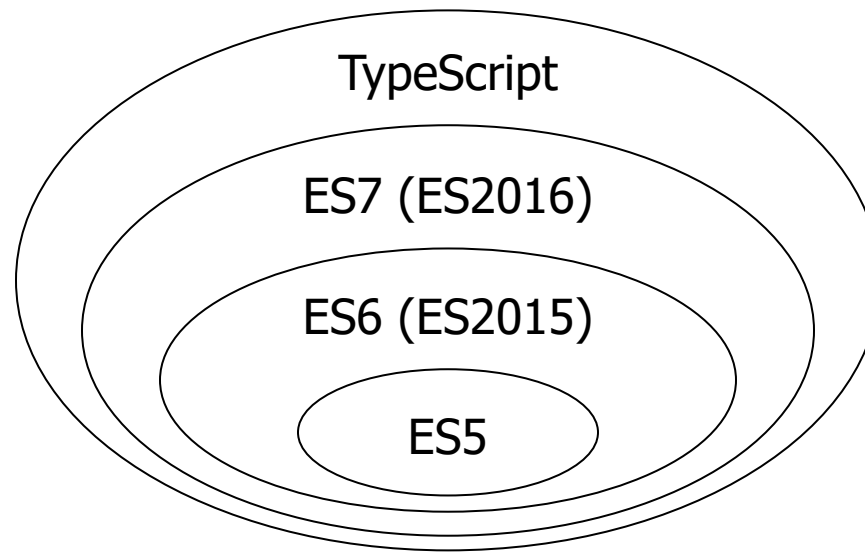


Angular CLI

- A command-line tool to create, build, and run Angular applications
- For Installation
 - `npm install -g @angular/cli`
- For Creating Project
 - `ng new <proj>`
- For run the app
 - `ng serve --open`

Angular - TypeScript

TypeScript



- JavaScript with Type
- Makes JavaScript more suitable for large software projects
 - Static type checking, improved tool support ...
- Developed by Microsoft
- Can be compiled into plain JavaScript

Compile and Run TypeScript

- Install NPM package `typescript`
 - `npm install -g typescript`
 - Use `tsc` to compile `.ts` files to `.js`
- Install NPM package `ts-node`
 - Use `ts-node` to run `.ts` files directly

ES6 and ES7 Features

- Class (ES6 and ES7)
- Decorators (ES7)
 - Configurable decorators
- Module (ES6)

Class Example

- Constructor
- Static and instance properties
- Static and instance initializers
- Static and instance methods
- Inheritance

Decorators

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']})
```

- Similar to annotations in Java
- Add metadata and/or behavior
- Types of decorators: class, property, method, and parameter

Class Decorator Example

- A class decorator is a function with one argument: the class (or more accurately, the *constructor function*)
- A configurable decorator is a function that takes a config object and returns a regular decorator function
- Enable `experimentalDecorators` option in `tsconfig.json` or on command line
- For Example

```
@log('hello')  
class World {  
}
```

TypeScript-Specific Features

- Types and type inference
- Classes and interfaces
 - Optional properties
 - Access modifiers
 - Abstract class
 - Interface
- Generics

Basic Types

- Primitive types
 - string
 - number
 - boolean
 - null
 - undefined
- Object types
 - String
 - Number
 - Boolean

Primitive types are automatically converted to object types when necessary

Type Declaration

```
function factorial(n: number): number {  
    if (n < 0) throw new Error('Invalid argument');  
    let result: number = 1;  
    for (let i: number = 1; i <= n; ++i) result *= i;  
    return result;  
}
```

Other Commonly Used Types

- Array
- enum
- any
- void

```
let sizes: string[] = ['S', 'M', 'L'];
```

```
function sayHi(): void {  
  console.log('Hi!');  
}
```

```
enum Color {Red, Green, Blue}
```

```
let color: Color = Color.Red;
```

```
console.log(`color is ${color}`);  
console.log(`Blue is ${Color.Blue}`);
```

Type Inference

```
let a = 'hello'; // string
```

```
let b = 2.1; // number
```

```
let c = [1, 2, 3]; // number[]
```

```
let d = ['one', 'two', 3];    //??
```

Optional Properties

```
class Foo {  
    a: number;  
    b?: number;  
}
```

- With `strictNullChecks` compiler option, `a` cannot be null or undefined, `b` can be undefined but not null

Access Modifier in Class

- `public`: default
- `private`: can be accessed within class
- `protected`: can be accessed within class and subclasses
- `readonly`: like `const` for properties

Parameter Property

```
class Foobar {  
    public foo: number;  
    private bar: number;  
  
    constructor( foo: number, bar: number ) {  
        this.foo = foo;  
        this.bar = bar;  
    }  
}
```



*Access modifiers are
required for parameter
properties*

```
class Foobar {  
    constructor( public foo: number, private bar: number ) {}  
}
```

Abstract Class

```
abstract class Animal {  
  
    abstract makeSound(): void;  
  
    move(): void {  
        console.log("roaming the earth...");  
    }  
}
```


Interface Example

- Define interface
- Interface inheritance
- Interface implementation

The Need for Generics

- Generics are often used to ensure that the data in a collection is of the same type

```
function compare(a, b) : number {...}
```

```
class BinaryTreeNode {  
    value;  
    left : BinaryTreeNode;  
    right : BinaryTreeNode;  
}
```

Generic Function and Class

```
function compare<T>(a : T, b: T) : number {...}
```

```
class BinaryTreeNode<T> {  
    value : T;  
    left : BinaryTreeNode<T>;  
    right : BinaryTreeNode<T>;  
}
```

**Type
Parameter**

```
interface Shape {...}  
class Circle implements Shape {...}  
class Square implements Shape {...}
```

```
function area<T extends Shape>(t: T): number {...}
```

Using Generics

- Specify a type for the type parameter

```
result = compare<number>(10, 20);
```

```
root = new BinaryTreeNode<string>();
```

Angular : Modules, Components & Directives

Example: Guest Book (Single Page)

My Guest Book		
		Add Comment
John says:	Hello!	4/29/2018
Jane says:	Your website looks nice.	4/11/2018
Joe says:	Nice to meet you.	3/5/2018

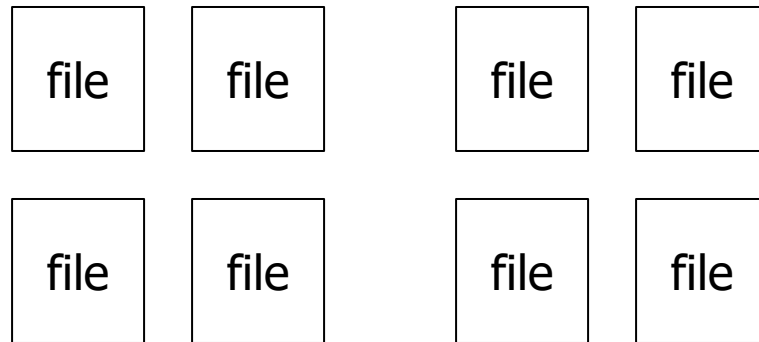
index.html

- The "single page" in SPA
- The page that hosts all the *components* such as **<app-root>**

```
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>Cs5220GuestbookClientAngular</title>
6    <base href="/">
7
8    <meta name="viewport" content="width=device-width, initial-scale=1">
9    <link rel="icon" type="image/x-icon" href="favicon.ico">
10 </head>
11 <body>
12   <app-root></app-root>
13 </body>
14 </html>
```

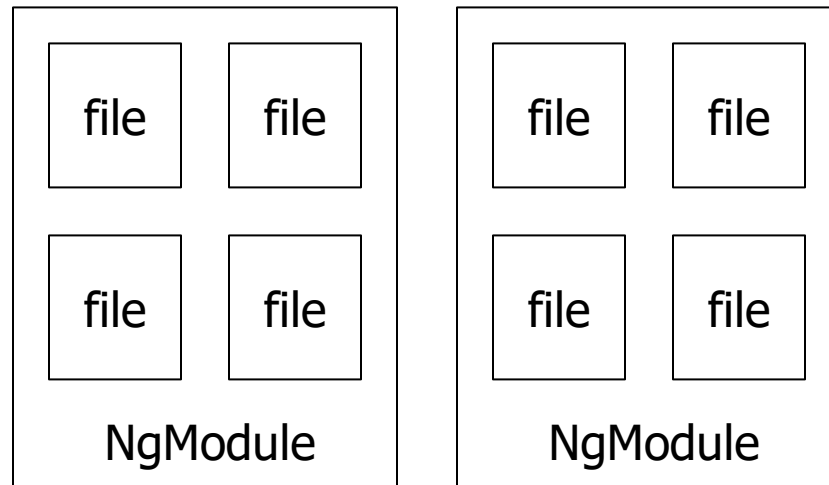
NgModule: The Angular Module System ...

- The one-file-per-module approach of CommonJS/ES6 is limiting



NgModule: The Angular Module System

- A NgModule group multiple files (i.e. components, services, and so on) into one module



@NgModule ...

- Each module must export a class that is decorated with `@NgModule`
- `declarations`: components/directives/pipes that belong to this module
- `imports`: modules which this module depends on
- `exports`: components/directives/pipes that can be used by other modules
- `providers`: injectable objects (usually services) provided by this module
- `bootstrap`: the component(s) to load when this module is used to bootstrap (i.e. to start) an application

About NgModule

- `declarations` and `exports` deal with *components/directives/pipes*, i.e. things used in templates
 - components/directives/pipes can be "private"
 - services are always "public"
- `imports` deals with modules
- `imports` vs `import`

About NgModule

src/app/app.module.ts

```
import { NgModule }      from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

@NgModule({
  imports:      [ BrowserModule ],
  providers:    [ Logger ],
  declarations: [ AppComponent ],
  exports:      [ AppComponent ],
  bootstrap:    [ AppComponent ]
})
export class AppModule { }
```

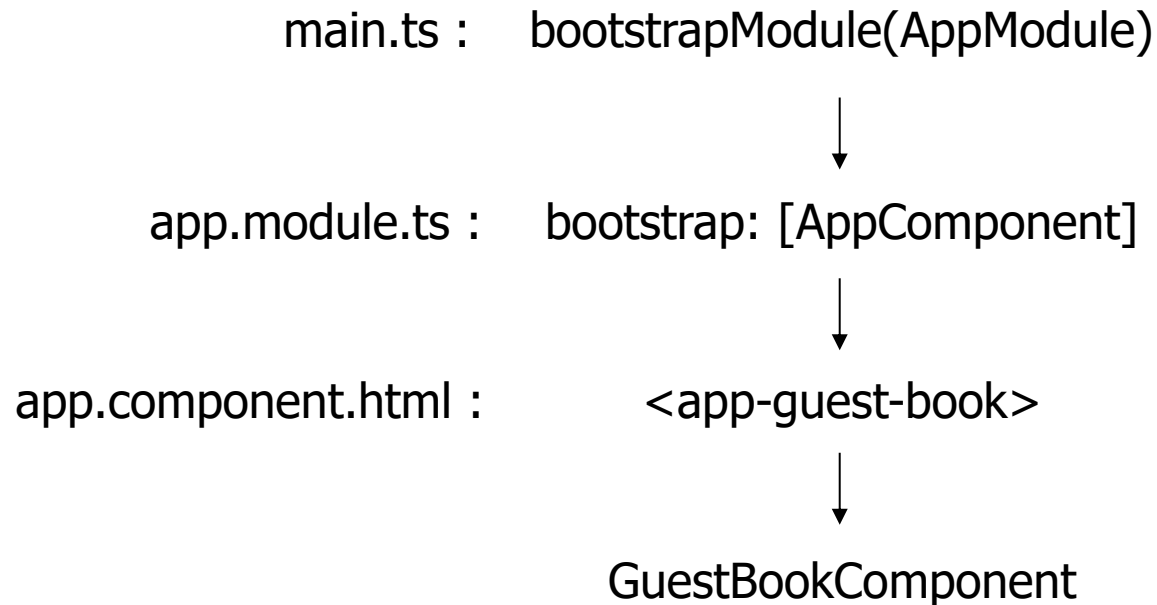
src/app/app.module.ts

```
import { NgModule }      from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent }  from './app.component';

@NgModule({
  imports:      [ BrowserModule ],
  declarations: [ AppComponent ],
  bootstrap:    [ AppComponent ]
})
export class AppModule { }
```

```
1  import { BrowserModule } from '@angular/platform-browser';
2  import { NgModule } from '@angular/core';
3  import { HttpClientModule } from '@angular/common/http';
4  import { Routes, RouterModule } from '@angular/router';
5
6  import { AppComponent } from './app.component';
7  import { GuestBookComponent } from './guest-book/guest-book.component';
8  import { AddEntryComponent } from './add-entry/add-entry.component';
9
10 const routes: Routes = [
11   { path: '', component: GuestBookComponent },
12   { path: 'AddEntry', component: AddEntryComponent }
13 ];
14
15 @NgModule({
16   declarations: [AppComponent, GuestBookComponent, AddEntryComponent],
17   imports: [BrowserModule, HttpClientModule, RouterModule.forRoot(routes)],
18   providers: [],
19   bootstrap: [AppComponent]
20 })
21 export class AppModule { }
```

Bootstrap An Application



Component

- User-defined tags as opposed to standard HTML tags like `<button>`
 - Appearance: HTML and CSS
 - Behavior: TypeScript
- **@ Component**
 - `selector`: name of the tag
 - `templateUrl` **or** `template`
 - `styleUrls` **or** `styles`
 - More options

Components - Example

```
1 import { Component, OnInit, Output, EventEmitter } from '@angular/core';
2 import { GuestBookEntry } from 'src/models/GuestBookEntry';
3 import { DataService } from '../data.service';
4 import { Router } from '@angular/router';
5
6 @Component({
7   selector: 'app-add-entry',
8   templateUrl: './add-entry.component.html',
9   styles: []
10 })
11 export class AddEntryComponent implements OnInit {
12   constructor(private dataService: DataService, private router: Router) {}
13
14   ngOnInit() {}
15
16   addEntry(name: HTMLInputElement, message: HTMLInputElement) {
17     this.dataService.addEntry(new GuestBookEntry(name.value, message.value));
18     this.router.navigate(['']);
19   }
20 }
```

```
1 import { Component, OnInit, Input } from '@angular/core';
2 import { GuestBookEntry } from 'src/models/GuestBookEntry';
3 import { DataService } from '../data.service';
4
5 @Component({
6   selector: 'app-guest-book',
7   templateUrl: './guest-book.component.html',
8   styles: []
9 })
10 export class GuestBookComponent implements OnInit {
11   entries: GuestBookEntry[];
12
13   constructor(private dataService: DataService) {}
14
15   ngOnInit() {
16     this.dataService.getEntries().subscribe(entries => {
17       this.entries = entries;
18     });
19   }
20 }
```

```
1 import { Component } from '@angular/core';
2 import { GuestBookEntry } from 'src/models/GuestBookEntry';
3
4 @Component({
5   selector: 'app-root',
6   templateUrl: './app.component.html',
7   styles: []
8 })
9 export class AppComponent {
10   title = 'cs5220-guestbook-client-angular';
11 }
```

Create A New Component

guest-book

- ng generate component <name>
- Generated files
- Naming conventions for components
 - Tags and files
 - Classes and methods

guest-book.component.html

```
1 <table>
2   <tr><th>Name</th><th>Message</th><th>Date</th></tr>
3   <tr *ngFor="let entry of entries">
4     <td>{{entry.name}}</td>
5     <td>{{entry.message}}</td>
6     <td>{{entry.date | date: 'M/d/yyyy'}}</td>
7   </tr>
8 </table>
9 <p><a routerLink="AddEntry">Add Entry</a></p>
```

guest-book.component.ts

```
1 import { Component, OnInit, Input } from '@angular/core';
2 import { GuestBookEntry } from 'src/models/GuestBookEntry';
3 import { DataService } from '../data.service';
4
5 @Component({
6   selector: 'app-guest-book',
7   templateUrl: './guest-book.component.html',
8   styles: []
9 })
10 export class GuestBookComponent implements OnInit {
11   entries: GuestBookEntry[];
12
13   constructor(private dataService: DataService) {}
14
15   ngOnInit() {
16     this.dataService.getEntries().subscribe(entries => {
17       this.entries = entries;
18     });
19   }
20 }
```


GuestBook – Data

- Data model class
- `ngOnInit()` is one of the component lifecycle hooks called *after* the component is constructed
- Code suggestion and auto import in Visual Studio Code

```
export class GuestBookEntry {  
    date = new Date();  
    constructor(public name: string, public message: string) {}  
}
```

```
export class GuestBookComponent implements OnInit {  
    entries: GuestBookEntry[];  
  
    constructor(private dataService: DataService) {}  
  
    ngOnInit() {  
        this.dataService.getEntries().subscribe(entries => {  
            this.entries = entries;  
        });  
    }  
}
```

GuestBook – Display

- Properties of a component class can be accessed directly in the component template
- Interpolation: `{{ }}`
- Directive: `ngFor`
- Pipe: `date`

```
1 <table>
2   <tr><th>Name</th><th>Message</th><th>Date</th></tr>
3   <tr *ngFor="let entry of entries">
4     <td>{{entry.name}}</td>
5     <td>{{entry.message}}</td>
6     <td>{{entry.date | date: 'M/d/yyyy'}}</td>
7   </tr>
8 </table>
9 <p><a routerLink="AddEntry">Add Entry</a></p>
```

Directive

- Angular templates are *dynamic*. When Angular renders them, it transforms the DOM according to the instructions given by **directives**.
 - Encapsulated *DOM-related logic* that can be used to compose user interface
- Types of directives
 - Component (i.e. directive with a view)
 - A component is a *directive-with-a-template*;
 - a `@Component` decorator is actually a `@Directive` decorator extended with template-oriented features.
 - [Structural directive](#), e.g. `ngIf`, `ngFor`, `ngSwitch`
 - [Attribute directive](#), e.g. `ngStyle` and `ngClass`

ngFor

```
<div *ngFor="let entry of entries">
  {{entry.name}}
</div>
```

Property



```
<div *ngFor="let entry of entries; let index = index">
  {{index}}. {{entry.name}}
</div>
```

Variable



- Variable can have any name
- Properties: `index`, `first`, `last`, `even`, `odd`

ngIf

```
<p *ngIf="entries.length == 0">  
  No entries yet.  
</p>
```

```
<p *ngIf="entries.length == 0; else hasEntries">  
  No entries yet.  
</p>
```

↑
Name of a
element

```
<ng-template #hasEntries>  
  <p>There are some entries.</p>  
</ng-template>
```

ngSwitch

```
<div [ngSwitch]="day">  
  <span *ngSwitchCase="1">Monday</span>  
  <span *ngSwitchCase="2">Tuesday</span>  
  <span *ngSwitchCase="3">Wednesday</span>  
  <span *ngSwitchCase="4">Thursday</span>  
  <span *ngSwitchCase="5">Friday</span>  
  <span *ngSwitchDefault>Weekend</span>  
</div>
```

ngStyle

An object



```
<p [ngStyle]="{color: 'red', background: 'yellow'}">
```

Some text

```
</p>
```

```
<p [style.color]="blue">hello</p>
```



A string

ngClass

CSS classes:

```
.red { color: red; }  
.yellow { background: yellow; }
```

```
<p [ngClass]="red">Some text</p>
```

```
<p [ngClass]="['yellow', 'red']">Some text</p>
```

```
<p [ngClass]="{yellow: true, red: false}">Some text</p>
```

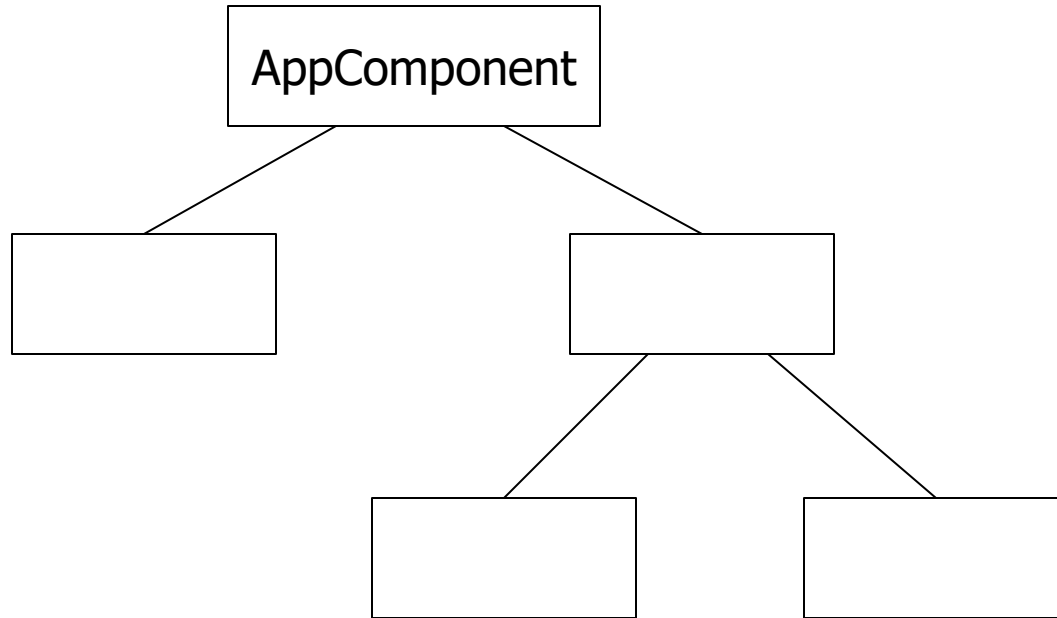

Pipes

- Chain-able formatting operators that can be applied to expressions

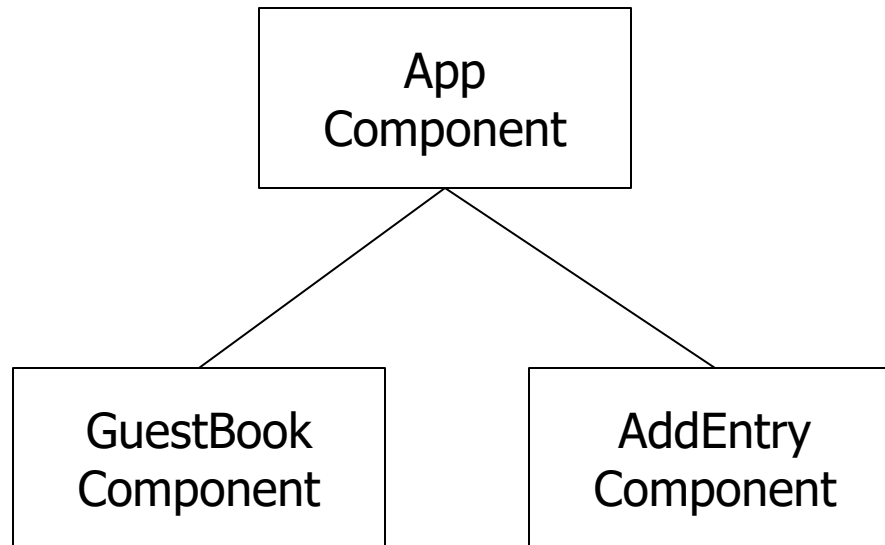
```
{{entry.date | date:'M/d/yyyy'}}
```

```
{{3.1415 | number:'1.2-2' | currency:'USD'}}
```

The Component Hierarchy



Example



Pass Data from Parent to Child

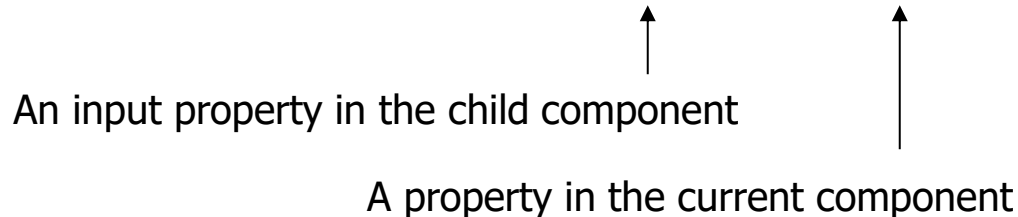
guest-book.component.ts:

```
@Input() entries: GuestBookEntry[];
```

app.component.html:

```
<app-guest-book [entries]="entries">
```

An input property in the child component



A property in the current component

AddEntryComponent Template

A variable in the view representing
the current element



Name: <input type="text" #name>

Message: <input type="text" #message>

<button (click)="addEntry(name, message)">Add</button>



Event



Event handler in the component

AddEntryComponent Code

- Angular is still JavaScript
 - DOM events like `click`
 - DOM elements like `HTMLInputElement`
- Event handler
 - `return false` to stop event propagation

```
1 <p><a routerLink="">GuestBook</a></p>
2 <p>
3   Name: <input type="text" #name> <br />
4   Message: <input type="text" #message> <br />
5   <button (click)="addEntry(name, message)">Add</button>
6 </p>
```

```
1 import { Component, OnInit, Output, EventEmitter } from '@angular/core';
2 import { GuestBookEntry } from 'src/models/GuestBookEntry';
3 import { DataService } from '../data.service';
4 import { Router } from '@angular/router';
5
6 @Component({
7   selector: 'app-add-entry',
8   templateUrl: './add-entry.component.html',
9   styles: []
10 })
11 export class AddEntryComponent implements OnInit {
12   constructor(private dataService: DataService, private router: Router) {}
13
14   ngOnInit() {}
15
16   addEntry(name: HTMLInputElement, message: HTMLInputElement) {
17     this.dataService.addEntry(new GuestBookEntry(name.value, message.value));
18     this.router.navigate(['']);
19   }
20 }
```

Pass Data from Child to Parent ...

add-entry.component.ts:

```
@Output() entryAdded: EventEmitter<GuestBookEntry>  
  = new EventEmitter<GuestBookEntry>();
```

```
this.entryAdded.emit(  
  new GuestBookEntry(name.value, message.value));
```

... Pass Data from Child to Parent

app.component.html:

```
<app-add-entry (entryAdded)="addEntry($event)">
```

app.component.ts:

```
addEntry(entry: GuestBookEntry): void {  
    this.entries.push(entry);  
}
```

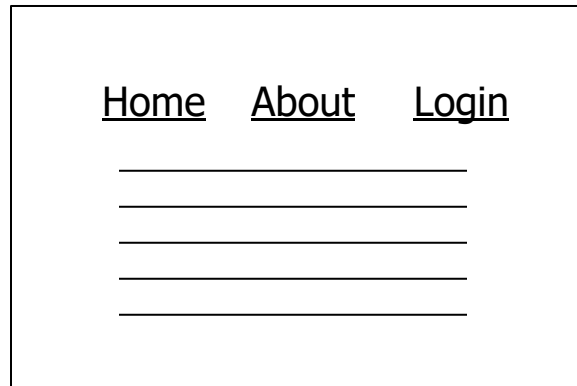

To Component Or Not To Component

- Putting everything inside one component seems to make coding easier (i.e. no need to pass data between components), so when is it better to create more components??

Angular: Routing

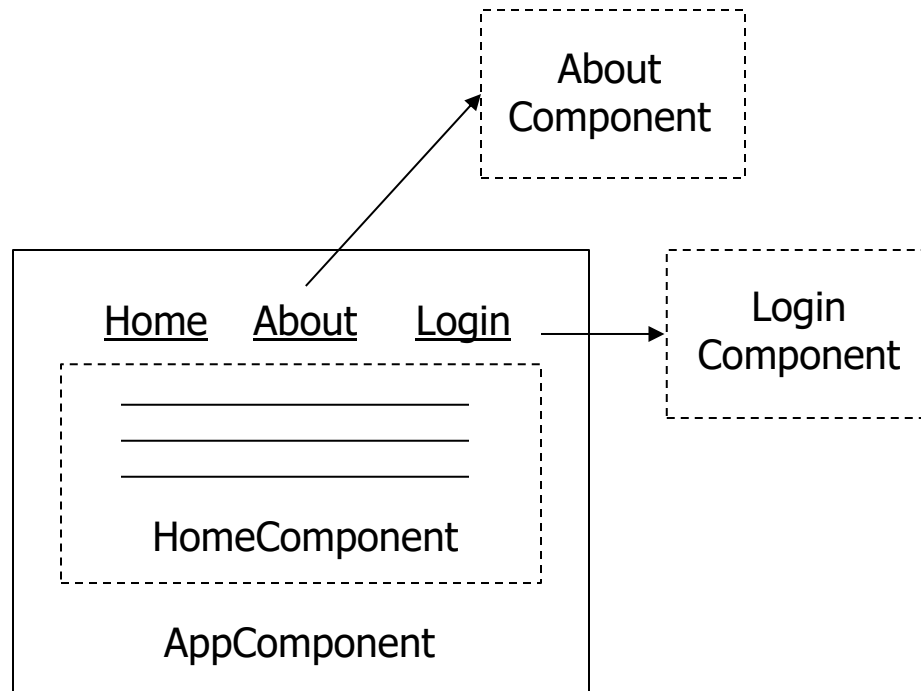
The Need for Routing

- Usually we have different pages in a web application
- How do we have different "pages" in a SPA?



Angular Routing

- Load different components based on different URL



Add A Routing Module

- The `routing` option of `ng new`
 - Add an `AppRoutingModule`
 - Import the routing module in `AppModule`
 - Add a `<router-outlet>` in App component template, which is a placeholder for the components to be loaded.

AppRoutingModule

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

const routes: Routes = [];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Routes

- Routes specify the mappings between paths and components, e.g.

```
const routes: Routes = [  
  { path: 'about', component: AboutComponent },  
  { path: 'login', component: LoginComponent }  
];
```

RouterModule

- `RouterModule.forRoot(routes)` returns a `RouterModule` that is configured with routes
- `AppRoutingModule` imports this module, then exports it so the rest of the application can use it
- You can specify routing directly in `AppModule`, but using an `AppRoutingModule` is recommended (for centralizing route management in large applications)

The forRoot() Convention

- `forRoot()` returns a module that should be imported by the root module (i.e. the module used for bootstrapping the application)
- Use `forChild()` if you want to add routing to a feature module

Basic Routing

```
const routes: Routes = [  
  { path: 'block1', component: Block1Component },  
  { path: 'block2', component: Block2Component }  
];
```

In template:

```
<a routerLink="block1">Block 1</a>  
<a routerLink="block2">Block 2</a>
```

Why routerLink instead of href?

- See the difference using Chrome's Developer Tools (Network tab)
- Clicking on a `routerLink` does NOT send a request – the address URL is changed through HTML5 history API

routerLink as @Input

Link is hard-coded in template



```
<a routerLink="block1">Block 1</a>
```

```
<a [routerLink]="\"block1\">Block 1</a>
```

```
<a [routerLink]="['block1']">Block 1</a>
```



Allow setting link with dynamic data from code

Route Segments

`<a routerLink="user/bob">Bob</a>`

`<a [routerLink]="user/bob">Bob</a>`

`<a [routerLink]="['user', 'bob']">Bob</a>`

`<a [routerLink]="['user', name]">{{name}}</a>`

Wildcard Route

```
const routes: Routes = [  
  { path: 'about', component: AboutComponent },  
  { path: 'login', component: LoginComponent },  
  { path: '**', component: PageNotFoundComp }  
];
```

- Catch un-matched routes and display a PageNotFound page (i.e. component)
- Route order matters so make sure the wildcard route is listed last

Navigate to Route in Code

- Example: click a button in Block1 to go to Block2

```
class Block1Component {  
    constructor(private router: Router) {}  
  
    goToBlock2(): boolean {  
        this.router.navigate( ['block2'] );  
        return false;  
    }  
}
```

Attach Information To Route

- Path parameters, e.g. `/user/:id`
- Matrix parameters, e.g.
`/user;fn=John;ln=Doe`
- Query parameters, e.g. `/user?id=`
- Path fragment, e.g. `/user#profile`

Specify Route Parameters

```
const routes: Routes = [  
  { path: 'block1/:id', component: Block1Component }  
];
```

In template:

```
<a routerLink="block1/1">Block 1</a>  
<a [routerLink]="['block1', 1]">Block 1</a>
```

In component class:

```
this.router.navigate(['block1', 1]);
```

Specify Matrix Parameters

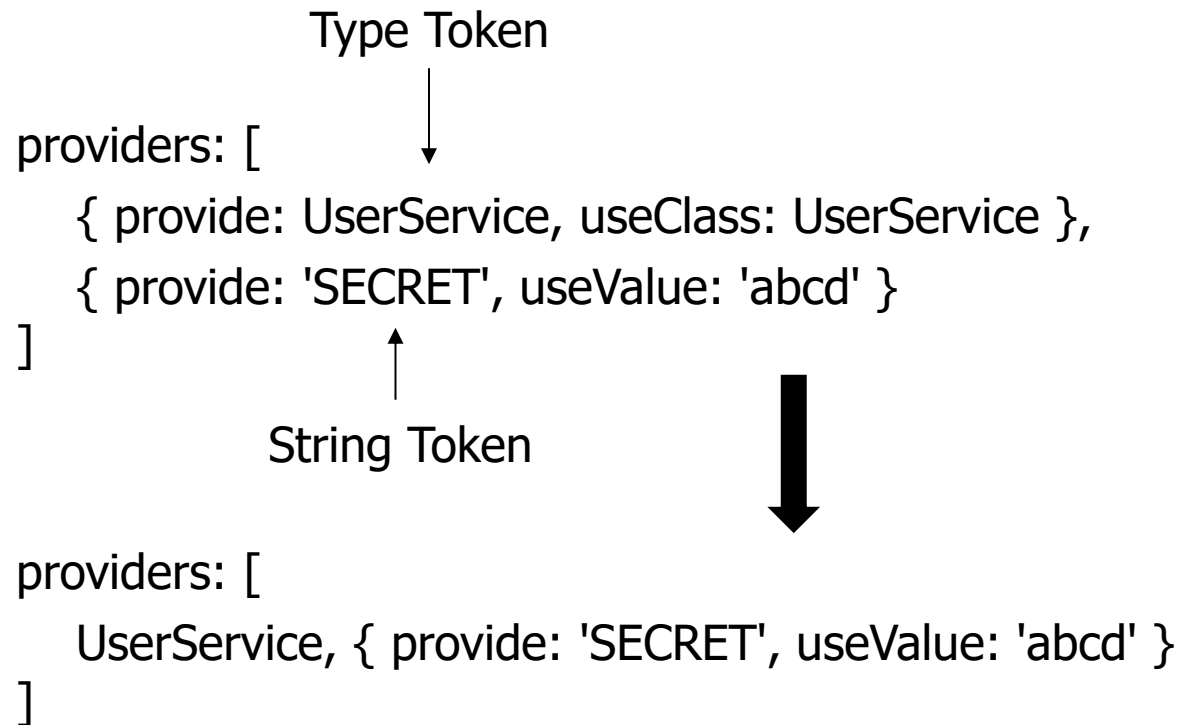
```
<a [routerLink]="['block1',{x:10, y:20}]">
```

```
  this.router.navigate([ 'block1',  
    {x:10, y:20}  
  ]);
```

Dependency Injection

- Things to inject: services and data
- Declare in `providers` of `NgModule`
- Inject in constructors

Dependency Injection: Providers



Dependency Injection: Inject

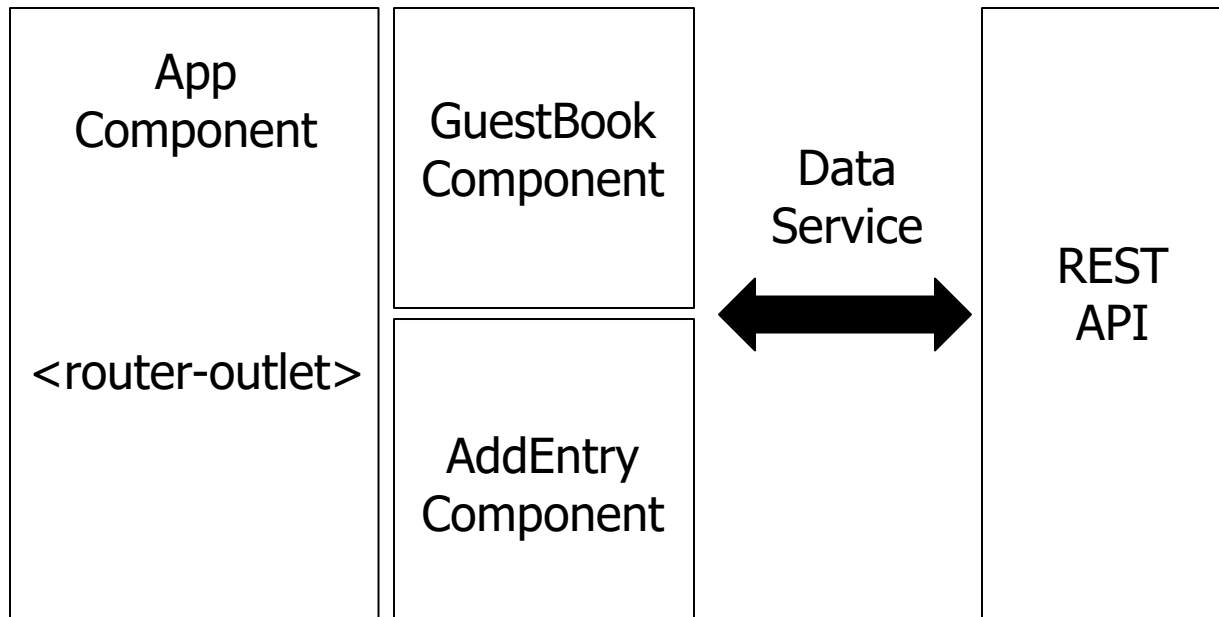
```
Class HomeComponent {  
    constructor(  
        private UserService: userService,  
        @Inject('SECRET') private secret: string  
    ) {}  
}
```

Angular : Services

Angular Services

- Implement business logic of an application
- Made available to other parts of the application via *dependency injection*
 - Decorated with `@Injectable`
 - Declared in `providers` of `@NgModule`
 - Injected into constructors

Example: GuestBook



Changes to AddEntry Form ...

Name: <input type="text" #name>

Message: <input type="text" #message>

<button (click)="addEntry(name,message)">

Add

</button>

... Changes to AddEntry Form

Name: `<input type="text" name="name"
[(ngModel)]="entry.name">`

Message: `<input type="text" name="message"
[(ngModel)]="entry.message">`

`<button (click)="addEntry()">Add</button>`

- **`[(ngModel)]` creates a 2-way binding between a form field and a property in the component class**

guest-book.component.ts

```
1 import { Component, OnInit, Input } from '@angular/core';
2 import { GuestBookEntry } from 'src/models/GuestBookEntry';
3 import { DataService } from '../data.service';
4
5 @Component({
6   selector: 'app-guest-book',
7   templateUrl: './guest-book.component.html',
8   styles: []
9 })
10 export class GuestBookComponent implements OnInit {
11   entries: GuestBookEntry[];
12
13   constructor(private dataService: DataService) {}
14
15   ngOnInit() {
16     this.dataService.getEntries().subscribe(entries => {
17       this.entries = entries;
18     });
19   }
20 }
```

GuestBookEntry.ts

```
1 export class GuestBookEntry {
2   date = new Date();
3   constructor(public name: string, public message: string) {}
4 }
```

data.service.ts

```
1 import { Injectable } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable, BehaviorSubject, observable } from 'rxjs';
4 import { GuestBookEntry } from 'src/models/GuestBookEntry';
5
6 @Injectable({
7   providedIn: 'root'
8 })
9 export class DataService {
10   private _entries: GuestBookEntry[] = [];
11   private _entriesSubject = new BehaviorSubject<GuestBookEntry[]>(
12     this._entries
13   );
14
15   constructor(private http: HttpClient) {
16     this.http.get<GuestBookEntry[]>('/api/guestbook').subscribe(entries => {
17       this._entries = entries;
18       this._entriesSubject.next(this._entries);
19     });
20   }
21
22   getEntries(): Observable<GuestBookEntry[]> {
23     return this._entriesSubject;
24   }
25
26   addEntry(entry: GuestBookEntry) {
27     this.http
28       .post<GuestBookEntry>('/api/guestbook', entry)
29       .subscribe(newEntry => {
30         this._entries.push(newEntry);
31         this._entriesSubject.next(this._entries);
32       });
33   }
34 }
```

Create A Remote Data Service

- Use Angular CLI to generate a service
 - Naming conventions
- Add it to AppModule
- Inject it to component classes

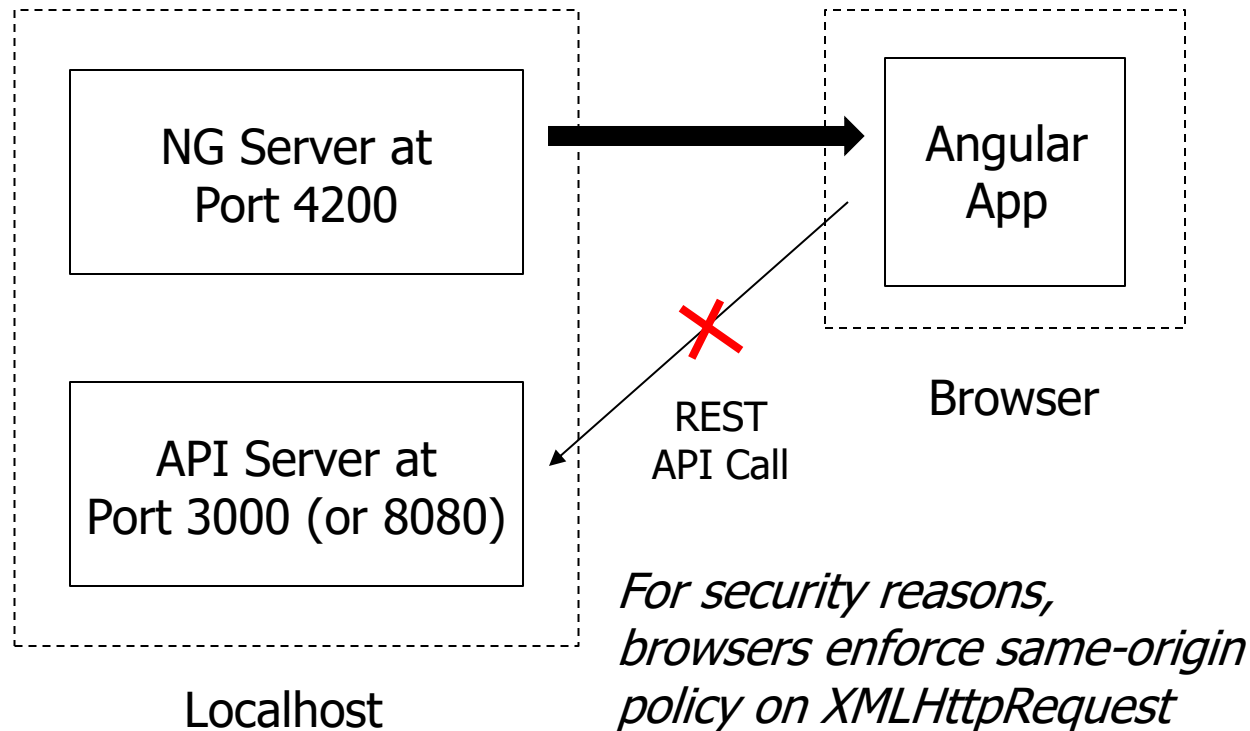
Use HttpClient to Access REST API

- **Import** `HttpClientModule` from `@angular/common/http`
- **Inject** `HttpClient` to code that needs to make API calls
- Use `HttpClient` API, e.g. [get](#)

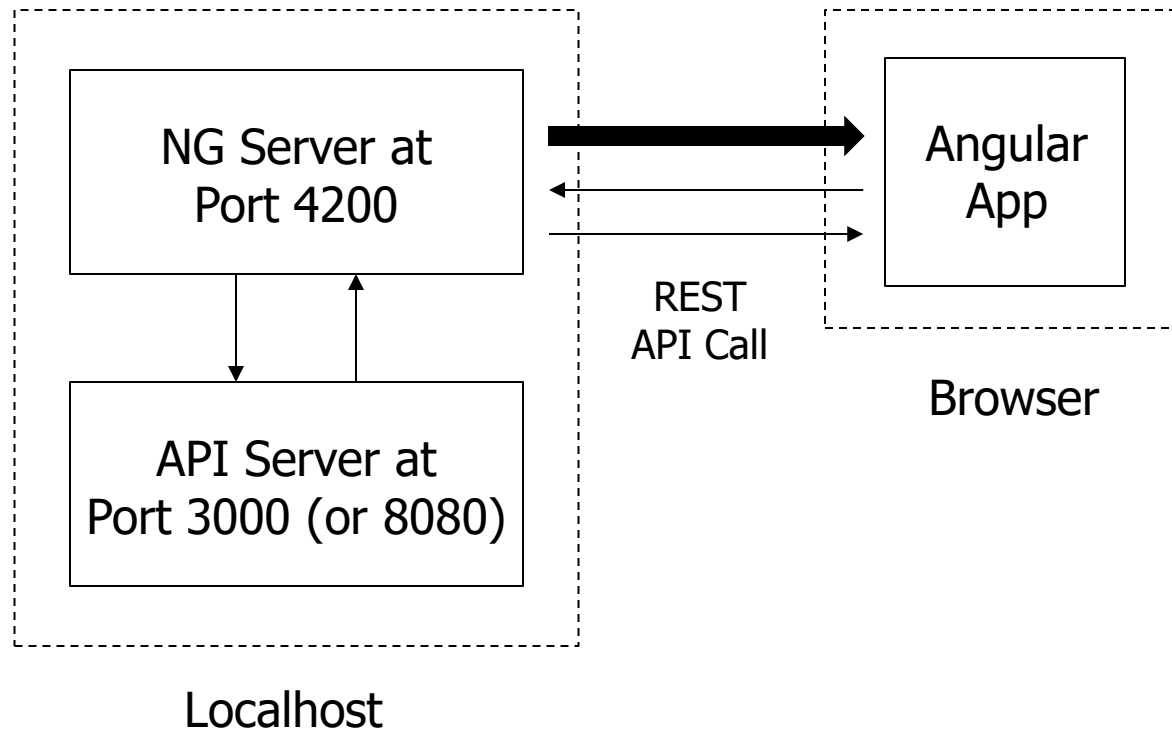


Set Up Proxy During Development

...



... Set Up Proxy During Development ...



... Set Up Proxy During Development

- Create proxy configuration file `proxy.conf.json`, e.g.

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false
  }
}
```

- For Connect to Express API, change `start` script in `package.json`

```
ng serve --proxy-config proxy.conf.json
```


Deploy Angular App

- Build Angular App: `ng build --prod`
 - The `/dist` folder will contain the files to be deployed
- Because Angular is just JavaScript, HTML, and CSS, an Angular app can be served as static resources as part of a web application or service

Quiz # 4

- Take input student's name, rollno, and age from HTML. When a user clicks on add button, this information should be added to the in-memory student's list after checking data validity.
 - The validity check includes proper name, rollno, and age within a specified range.
 - No DB manipulation is required.

Thank you!



National University
Of Computer and Emerging Sciences



quest lab